

ML Final Exam - Question & Model Answer Reference

This reference covers [week8.ipynb](#) to [week13.ipynb](#): tree methods, SVMs, PCA, clustering, EM/GMM, and neural networks.

TOPIC 1 - Classification Tree (Week 8)

Dataset: [data/carseats.csv](#)

Create `High = 1` if `Sales > 8`, otherwise `0`. Fit a classification tree to predict `High` and evaluate it on a test set.

Model Answer:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix, accuracy_score,
classification_report

df = pd.read_csv('data/carseats.csv')
df['High'] = (df['Sales'] > 8).astype(int)
df['ShelveLoc'] = pd.factorize(df['ShelveLoc'])[0]
df['Urban'] = df['Urban'].map({'No': 0, 'Yes': 1})
df['US'] = df['US'].map({'No': 0, 'Yes': 1})

X = df.drop(['Sales', 'High'], axis=1)
y = df['High']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, train_size=0.8, random_state=0
)

tree = DecisionTreeClassifier(max_depth=6, random_state=0)
tree.fit(X_train, y_train)

pred = tree.predict(X_test)
print(confusion_matrix(y_test, pred))
print('Test accuracy:', accuracy_score(y_test, pred))
print(classification_report(y_test, pred))

importance = pd.Series(tree.feature_importances_, index=X.columns)
print(importance.sort_values(ascending=False))
```

Interpretation: A classification tree predicts the majority class in each terminal node. The confusion matrix shows correct and incorrect predictions. Important variables are those that reduce node impurity strongly, often `Price` or `ShelveLoc` in the Carseats data.

TOPIC 2 - Regression Tree (Week 8)

Dataset: `data/boston.csv`

Fit a regression tree to predict `medv`. Print test MSE, RMSE, and feature importances.

Model Answer:

```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeRegressor
from sklearn.metrics import mean_squared_error

df = pd.read_csv('data/boston.csv')
X = df.drop('medv', axis=1)
y = df['medv']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

tree = DecisionTreeRegressor(max_depth=4, random_state=1)
tree.fit(X_train, y_train)

pred = tree.predict(X_test)
mse = mean_squared_error(y_test, pred)
print('Test MSE:', mse)
print('Test RMSE:', np.sqrt(mse))

importance = pd.Series(tree.feature_importances_, index=X.columns)
print(importance.sort_values(ascending=False).head())
```

Interpretation: A regression tree predicts the average response in the terminal node. RMSE is easier to interpret than MSE because it is in the same unit as the response.

TOPIC 3 - Tree Pruning Concept (Week 8)

Explain why a decision tree should be pruned and describe cost complexity pruning.

Model Answer:

A very large tree can overfit: it may explain the training data very well but generalize poorly to new data. Pruning grows a large tree first, then cuts it back to a smaller subtree.

Cost complexity pruning balances training error and tree size:

$$\text{RSS} + \alpha * |T|$$

where $|T|$ is the number of terminal nodes and α is the tuning parameter. If $\alpha = 0$, large trees are not penalized. As α increases, larger trees are penalized more, so smaller trees are selected.

Interpretation: Pruning usually reduces variance. In practice, choose α using cross-validation.

TOPIC 4 - Bagging vs Random Forest (Week 8)

Dataset: `data/boston.csv`

Use `RandomForestRegressor` to fit bagging and random forest models. Compare test MSE.

Model Answer:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor
from sklearn.metrics import mean_squared_error

df = pd.read_csv('data/boston.csv')
X = df.drop('medv', axis=1)
y = df['medv']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

p = X.shape[1]

bag = RandomForestRegressor(n_estimators=500, max_features=p, random_state=1)
bag.fit(X_train, y_train)

rf = RandomForestRegressor(n_estimators=500, max_features=6, random_state=1)
rf.fit(X_train, y_train)

print('Bagging MSE:', mean_squared_error(y_test, bag.predict(X_test)))
print('Random forest MSE:', mean_squared_error(y_test, rf.predict(X_test)))

importance = pd.Series(rf.feature_importances_, index=X.columns)
print(importance.sort_values(ascending=False).head())
```

Interpretation: Bagging averages many bootstrap trees. Random forests are like bagging, but each split only considers a random subset of predictors. This decorrelates trees and often improves test performance.

TOPIC 5 - Boosting (Week 8)

Dataset: `data/boston.csv`

Fit boosted regression trees using 500 trees and compare two learning rates.

Model Answer:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import mean_squared_error

df = pd.read_csv('data/boston.csv')
X = df.drop('medv', axis=1)
y = df['medv']

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=1
)

for lr in [0.01, 0.2]:
    model = GradientBoostingRegressor(
        n_estimators=500,
        learning_rate=lr,
        max_depth=4,
        random_state=1
    )
    model.fit(X_train, y_train)
    pred = model.predict(X_test)
    print(f'Learning rate {lr}, test MSE:', mean_squared_error(y_test, pred))
```

Interpretation: Boosting grows trees sequentially. Each new tree improves the current model, often by fitting residuals. Small learning rate means slower learning and usually needs more trees. Large learning rate can overfit.

TOPIC 6 - Compare Tree-Based Methods (Week 8)

Compare a single tree, bagging, random forest, and boosting.

Model Answer:

Method	Main idea	Strength	Weakness
Single tree	One recursive tree	Very interpretable	High variance
Bagging	Average bootstrap trees	Reduces variance	Trees can be correlated
Random forest	Bagging plus random feature subset	Lower tree correlation	Less interpretable
Boosting	Sequential trees improve errors	Often very accurate	Needs careful tuning

Interpretation: Single trees are easiest to explain. Ensembles usually predict better but sacrifice interpretability.

TOPIC 7 - Linear Support Vector Classifier (Week 9)

Generate simulated two-class data, fit a linear SVC, and tune C by cross-validation.

Model Answer:

```
import numpy as np
from sklearn.svm import SVC
from sklearn.model_selection import GridSearchCV
from sklearn.metrics import confusion_matrix, accuracy_score

np.random.seed(5)
X = np.random.randn(20, 2)
y = np.repeat([1, -1], 10)
X[y == -1] = X[y == -1] + 2

params = {'C': [0.001, 0.01, 0.1, 1, 5, 10, 100]}
grid = GridSearchCV(SVC(kernel='linear'), params, cv=10, scoring='accuracy')
grid.fit(X, y)

print('Best C:', grid.best_params_)
print('Best CV score:', grid.best_score_)

svc = grid.best_estimator_
pred = svc.predict(X)
print(confusion_matrix(y, pred))
print('Training accuracy:', accuracy_score(y, pred))
print('Number of support vectors:', svc.support_.size)
```

Interpretation: C controls the cost of margin violations. Small C gives a wider margin and usually more support vectors. Large C gives a narrower margin and penalizes violations more.

TOPIC 8 - Maximal Margin vs Soft Margin (Week 9)

Explain the difference between a maximal margin classifier and a support vector classifier.

Model Answer:

The maximal margin classifier requires perfectly linearly separable data. It finds the separating hyperplane with the largest margin.

The support vector classifier allows some observations to be inside the margin or even on the wrong side of the hyperplane. These are margin violations controlled by slack variables and the tuning parameter C .

```
Maximal margin: perfect separation required.
Soft margin SVC: allows violations, usually better for real data.
Support vectors: observations on or inside the margin; they determine the
boundary.
```

Interpretation: Real data are rarely perfectly separable, so the soft margin classifier is usually more practical.

TOPIC 9 - Radial Kernel SVM (Week 9)

Generate nonlinear data and fit an RBF SVM. Tune C and γ .

Model Answer:

```
import numpy as np
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split, GridSearchCV
from sklearn.metrics import confusion_matrix, accuracy_score

np.random.seed(1)
X = np.random.randn(200, 2)
X[:100] = X[:100] + 2
X[100:150] = X[100:150] - 2
y = np.repeat([1, -1], 100)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=1
)

params = {'C': [0.1, 1, 10, 100], 'gamma': [0.5, 1, 2, 3]}
grid = GridSearchCV(SVC(kernel='rbf'), params, cv=5, scoring='accuracy')
grid.fit(X_train, y_train)

print('Best parameters:', grid.best_params_)
pred = grid.best_estimator_.predict(X_test)
print(confusion_matrix(y_test, pred))
print('Test accuracy:', accuracy_score(y_test, pred))
```

Interpretation: The radial kernel creates nonlinear boundaries. γ controls how local the influence of each point is. High γ can make the boundary very flexible and may overfit.

TOPIC 10 - SVM Kernel Concept (Week 9)

Explain linear, polynomial, and radial kernels.

Model Answer:

Kernel	Formula idea	Best use
Linear	Standard inner product	Roughly linear boundary
Polynomial	Adds polynomial feature effects	Curved polynomial boundary
Radial/RBF	Similarity decreases with distance	Flexible nonlinear boundary

Linear: $K(x_i, x_j) = \sum(x_i * x_j)$
 Polynomial: $K(x_i, x_j) = (1 + \sum(x_i * x_j))^d$

Radial: $K(x_i, x_j) = \exp(-\gamma \sum (x_i - x_j)^2)$

Interpretation: Kernels let SVMs act as if data were transformed into a higher-dimensional space without explicitly computing that transformation.

TOPIC 11 - ROC Curve for SVM (Week 9)

Fit two SVMs and compare them with ROC curves using `decision_function()`.

Model Answer:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.svm import SVC
from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_curve, auc

np.random.seed(1)
X = np.random.randn(200, 2)
X[:100] = X[:100] + 2
X[100:150] = X[100:150] - 2
y = np.repeat([1, 0], 100)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=1
)

for gamma in [0.5, 3]:
    model = SVC(kernel='rbf', gamma=gamma, C=1)
    model.fit(X_train, y_train)
    scores = model.decision_function(X_test)
    fpr, tpr, threshold = roc_curve(y_test, scores)
    roc_auc = auc(fpr, tpr)
    plt.plot(fpr, tpr, label=f'gamma={gamma}, AUC={roc_auc:.3f}')

plt.plot([0, 1], [0, 1], 'k--')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.legend()
plt.show()
```

Interpretation: ROC curves compare performance across thresholds. AUC near 1 is strong; AUC near 0.5 is random guessing. SVM class labels alone are not enough for ROC; use scores from `decision_function()`.

TOPIC 12 - PCA on USArrests (Week 10)

Dataset: `data/USArrests.csv` or `data/usarrests.csv`

Scale the data, run PCA, and print loadings plus variance explained.

Model Answer:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import scale
from sklearn.decomposition import PCA

df = pd.read_csv('data/USArrests.csv', index_col=0)
X = pd.DataFrame(scale(df), index=df.index, columns=df.columns)

pca = PCA()
scores = pca.fit_transform(X)

loadings = pd.DataFrame(
    pca.components_.T,
    index=df.columns,
    columns=[f'PC{i}' for i in range(1, X.shape[1] + 1)]
)

print(loadings)
print('PVE:', pca.explained_variance_ratio_)
print('Cumulative PVE:', np.cumsum(pca.explained_variance_ratio_))

plt.plot(range(1, 5), pca.explained_variance_ratio_, '-o')
plt.xlabel('Principal Component')
plt.ylabel('Proportion of Variance Explained')
plt.show()
```

Interpretation: PCA finds directions of maximum variance. Scaling is important because variables have different units and variances. Loadings are variable weights; scores are observation coordinates.

TOPIC 13 - PCA Interpretation (Week 10)

For USArrests, PC1 has large loadings for **Murder**, **Assault**, and **Rape**. PC2 has a large loading for **UrbanPop**. Interpret PC1 and PC2.

Model Answer:

PC1 represents an overall serious-crime direction. States with high PC1 scores tend to have high murder, assault, and rape rates.

PC2 represents urbanization because it is strongly related to **UrbanPop**. States with high PC2 scores tend to be more urban.

```
Loadings = weights for original variables.
Scores   = positions of observations on PCs.
```


PVE = proportion of total variance explained.

Interpretation: PCA is unsupervised. It is used for visualization, exploratory analysis, and dimension reduction, not direct prediction.

TOPIC 14 - PCA on NCI60 (Week 10)

Dataset: `data/nci60_data.csv`, `data/nci60_labs.csv`

Run PCA on gene expression data and plot PC1 vs PC2 colored by cancer type.

Model Answer:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.preprocessing import scale
from sklearn.decomposition import PCA

X_raw = pd.read_csv('data/nci60_data.csv', index_col=0)
labs = pd.read_csv('data/nci60_labs.csv', index_col=0)

X = scale(X_raw)
pca = PCA()
scores = pca.fit_transform(X)

color_id = pd.factorize(labs.iloc[:, 0])[0]
plt.scatter(scores[:, 0], scores[:, 1], c=color_id, cmap='hsv', alpha=0.7)
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.title('NCI60 PCA')
plt.show()

print(np.cumsum(pca.explained_variance_ratio_)[:10])
```

Interpretation: If samples of the same cancer type cluster together in the PC plot, their gene expression profiles are similar. In high-dimensional data, the first few PCs may explain only a moderate fraction of total variance.

TOPIC 15 - K-Means Clustering (Week 11)

Generate simulated data, run K-means with `K=2` and `K=3`, and compare inertia.

Model Answer:

```
import numpy as np
import matplotlib.pyplot as plt
```

```

from sklearn.cluster import KMeans

np.random.seed(123)
X = np.random.randn(50, 2)
X[:25, 0] = X[:25, 0] + 3
X[:25, 1] = X[:25, 1] - 4

for k in [2, 3]:
    km = KMeans(n_clusters=k, n_init=20, random_state=123)
    km.fit(X)
    print(f'K={k}')
    print('Inertia:', km.inertia_)
    print('Centers:\n', km.cluster_centers_)

    plt.scatter(X[:, 0], X[:, 1], c=km.labels_, cmap='prism')
    plt.scatter(km.cluster_centers_[0], km.cluster_centers_[1],
                marker='*', s=200, color='black')
    plt.title(f'K-means with K={k}')
    plt.show()

```

Interpretation: K-means needs K before fitting. Inertia is within-cluster sum of squares. It decreases as K increases, so do not choose huge K just because inertia is smaller.

TOPIC 16 - Manual K-Means Calculation (Week 11)

Given:

i	x1	x2	Initial cluster
1	2	2	A
2	3	4	A
3	8	8	B
4	9	6	B

Calculate centroids and perform one reassignment using Euclidean distance.

Model Answer:

Initial clusters:

```

A = 01, 02
B = 03, 04

```

Centroids:

$$CA = ((2 + 3) / 2, (2 + 4) / 2) = (2.5, 3)$$

$$CB = ((8 + 9) / 2, (8 + 6) / 2) = (8.5, 7)$$

Distance table:

i	d to CA	d to CB	New cluster
1	1.118	8.201	A
2	1.118	6.265	A
3	7.433	1.118	B
4	7.159	1.118	B

Interpretation: The assignments do not change, so K-means would stop.

TOPIC 17 - K-Modes for Categorical Data (Week 11)

Given observations:

i	x1	x2	x3
1	A	L	M
2	B	P	C
3	A	L	C
4	B	L	C
5	A	P	M

Initial modes:

$$C1 = (A, L, C)$$

$$C2 = (B, P, C)$$

Calculate dissimilarities and assignments.

Model Answer:

For K-modes:

```
same = 0
different = 1
lowest dissimilarity wins
```

i	Observation	d to C1	d to C2	Assignment
1	(A, L, M)	1	3	C1
2	(B, P, C)	2	0	C2
3	(A, L, C)	0	2	C1
4	(B, L, C)	1	1	tie
5	(A, P, M)	2	2	tie

Interpretation: K-modes is for categorical data. It uses modes instead of means. If a tie occurs, state the tie rule clearly.

TOPIC 18 - K-Means, K-Modes, K-Medoids, K-Prototypes (Week 11)

Which clustering method is suitable for numerical, categorical, mixed, and outlier-sensitive data?

Model Answer:

Data situation	Method	Reason
Numerical variables	K-means	Uses numerical centroids and distances
Categorical variables	K-modes	Uses modes and matching dissimilarity
Mixed numerical and categorical	K-prototypes	Combines K-means and K-modes ideas
Numerical data with outliers	K-medoids	Uses actual observations as centers

Interpretation: K-means is sensitive to outliers because means move toward extreme values. K-medoids is more robust because a medoid must be an actual observation.

TOPIC 19 - Hierarchical Clustering (Week 12)

Perform hierarchical clustering using complete, average, and single linkage. Cut the complete linkage tree into two clusters.

Model Answer:

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.cluster.hierarchy import linkage, dendrogram, cut_tree

np.random.seed(123)
X = np.random.randn(50, 2)
X[:25, 0] = X[:25, 0] + 3
X[:25, 1] = X[:25, 1] - 4

for method in ['complete', 'average', 'single']:
    hc = linkage(X, method=method)
```

```
plt.figure(figsize=(10, 5))
dendrogram(hc, leaf_rotation=90, leaf_font_size=8)
plt.title(f'{method} linkage')
plt.xlabel('Observation')
plt.ylabel('Distance')
plt.show()

hc_complete = linkage(X, method='complete')
clusters = cut_tree(hc_complete, n_clusters=2).reshape(-1)
print(clusters)
```

Interpretation: Hierarchical clustering does not require choosing the number of clusters before fitting. You choose clusters afterward by cutting the dendrogram. Single linkage can create chaining; complete and average linkage usually give more balanced clusters.

TOPIC 20 - Dendrogram Interpretation (Week 12)

How do you interpret a dendrogram?

Model Answer:

1. Each observation starts as its own cluster.
2. Similar clusters merge first.
3. The vertical height of a merge shows dissimilarity.
4. Lower merge height means more similar observations/clusters.
5. Horizontal position is mainly for display, not distance.

To choose clusters, draw a horizontal line across the dendrogram. The number of vertical branches crossed gives the number of clusters.

Interpretation: Read similarity from the vertical axis, not the horizontal spacing.

TOPIC 21 - Gaussian Mixture Model and EM (Week 12)

Fit a Gaussian mixture model and print posterior probabilities.

Model Answer:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture

np.random.seed(123)
X1 = np.random.randn(100, 2) + np.array([2, 2])
X2 = np.random.randn(100, 2) + np.array([-2, -2])
X = np.vstack([X1, X2])
```

```
gmm = GaussianMixture(n_components=2, covariance_type='full', random_state=123)
gmm.fit(X)

labels = gmm.predict(X)
probs = gmm.predict_proba(X)

print('Means:\n', gmm.means_)
print('First 5 labels:', labels[:5])
print('First 5 probabilities:\n', probs[:5])

plt.scatter(X[:, 0], X[:, 1], c=labels, cmap='viridis')
plt.title('Gaussian Mixture Model')
plt.show()
```

Interpretation: GMM is soft clustering. Each observation receives probabilities of belonging to each component. This is more flexible than hard assignment in K-means.

TOPIC 22 - EM Algorithm Concept (Week 12)

Explain the EM algorithm for Gaussian mixture models.

Model Answer:

EM is used when there are hidden variables. In a GMM, the hidden variable is the unknown component membership.

E-step:

Estimate the probability that each observation belongs to each Gaussian component.

M-step:

Update means, covariances, and mixing proportions using those probabilities.

Repeat until the likelihood stops improving.

Interpretation: K-means can be seen as hard EM. GMM uses soft probabilities and can model elliptical clusters.

TOPIC 23 - Choosing GMM Components with AIC/BIC (Week 12)

Use AIC and BIC to select the number of GMM components.

Model Answer:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.mixture import GaussianMixture
```

```

np.random.seed(123)
X1 = np.random.randn(100, 2) + np.array([2, 2])
X2 = np.random.randn(100, 2) + np.array([-2, -2])
X3 = np.random.randn(100, 2) + np.array([2, -2])
X = np.vstack([X1, X2, X3])

ks = range(1, 8)
aic, bic = [], []

for k in ks:
    model = GaussianMixture(n_components=k, covariance_type='full',
random_state=123)
    model.fit(X)
    aic.append(model.aic(X))
    bic.append(model.bic(X))

plt.plot(ks, aic, '-o', label='AIC')
plt.plot(ks, bic, '-s', label='BIC')
plt.xlabel('Number of components')
plt.ylabel('Criterion')
plt.legend()
plt.show()

print('Best AIC K:', list(ks)[np.argmin(aic)])
print('Best BIC K:', list(ks)[np.argmin(bic)])

```

Interpretation: Smaller AIC/BIC is preferred. Both balance model fit and complexity. BIC usually penalizes complexity more strongly than AIC.

TOPIC 24 - Neural Network Classifier (Week 13)

Fit an `MLPClassifier` to the Iris dataset. Encode the target, scale the predictors, and print performance metrics.

Model Answer:

```

import pandas as pd
from sklearn import preprocessing
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import confusion_matrix, classification_report

url = 'https://archive.ics.uci.edu/ml/machine-learning-databases/iris/iris.data'
names = ['sepal-length', 'sepal-width', 'petal-length', 'petal-width', 'Class']
iris = pd.read_csv(url, names=names)

X = iris.iloc[:, 0:4]
y = iris[['Class']]

```

```

y = y.apply(preprocessing.LabelEncoder().fit_transform)

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=1
)

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

mlp = MLPClassifier(hidden_layer_sizes=(10, 10, 10), max_iter=1000,
random_state=1)
mlp.fit(X_train, y_train.values.ravel())

pred = mlp.predict(X_test)
print(confusion_matrix(y_test, pred))
print(classification_report(y_test, pred))

```

Interpretation: Neural networks should usually use scaled inputs. Hidden layers allow nonlinear decision boundaries. One epoch is one full pass through the training data.

TOPIC 25 - MLP Regressor vs Linear Model (Week 13)

Create nonlinear synthetic data and compare Ridge regression with an MLP regressor.

Model Answer:

```

import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.linear_model import Ridge
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error, r2_score

np.random.seed(0)
N = 2000
X = 0.5 * np.random.normal(size=N) + 0.35
Xt = 0.75 * X - 0.35
X = X.reshape((N, 1))
y = -(8 * Xt**2 + 0.1 * Xt + 0.1) + 0.05 * np.random.normal(size=N)
y = np.exp(y) + 0.05 * np.random.normal(size=N)
y = y / max(abs(y))

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.5, random_state=0
)

ridge = Ridge().fit(X_train, y_train)
ridge_pred = ridge.predict(X_test)

```



```
mlp = MLPRegressor(hidden_layer_sizes=(16, 8), activation='tanh',
                    max_iter=1000, random_state=0)
mlp.fit(X_train, y_train)
mlp_pred = mlp.predict(X_test)

print('Ridge MSE:', mean_squared_error(y_test, ridge_pred))
print('Ridge R2:', r2_score(y_test, ridge_pred))
print('MLP MSE:', mean_squared_error(y_test, mlp_pred))
print('MLP R2:', r2_score(y_test, mlp_pred))

plt.scatter(X_test[:, 0], y_test, s=10, alpha=0.4, label='True')
plt.scatter(X_test[:, 0], mlp_pred, s=10, alpha=0.4, label='MLP')
plt.legend()
plt.show()
```

Interpretation: Ridge is linear, so it struggles with nonlinear patterns. An MLP with hidden layers can learn nonlinear relationships. ReLU gives piecewise linear fits; tanh/logistic activations are smoother.

TOPIC 26 - Neural Network Concepts (Week 13)

Explain feed-forward, backpropagation, activation function, loss function, epoch, learning rate, and momentum.

Model Answer:

Term	Meaning
Feed-forward	Inputs pass through layers to produce prediction
Activation function	Nonlinear function applied to weighted input
Loss function	Measures prediction error
Backpropagation	Computes gradients of loss with respect to weights
Gradient descent	Updates weights to reduce loss
Epoch	One full pass through the training data
Learning rate	Step size for weight updates
Momentum	Uses previous update direction to speed or stabilize learning

Basic neuron:

```
net_i = sum(w_ij * x_j) + b_i
activation_i = sigma(net_i)
```

Sigmoid:

$$\text{sigma}(x) = 1 / (1 + \exp(-x))$$

Interpretation: Neural networks learn by predicting, measuring error, and adjusting weights repeatedly.

TOPIC 27 - Activation and Output Choices (Week 13)

Choose suitable hidden and output activations.

Model Answer:

Problem	Hidden activation	Output activation
Binary classification	ReLU or tanh	Sigmoid
Multi-class classification	ReLU or tanh	Softmax
Regression	ReLU or tanh	Linear

Interpretation: Hidden layers usually need nonlinear activations. The output activation should match the response: probability for classification, continuous value for regression.

Final Exam Quick Checklist

- Trees split predictor space into rectangular regions.
- Regression trees predict terminal-node means.
- Classification trees predict terminal-node majority classes.
- Bagging reduces variance using bootstrap trees.
- Random forests decorrelate trees using random predictor subsets.
- Boosting builds trees sequentially and uses a learning rate.
- SVM C controls margin violation cost.
- SVM γ controls RBF boundary flexibility.
- PCA should usually be done after scaling.
- PCA loadings are variable weights; scores are observation coordinates.
- K-means uses means and numerical distance.
- K-modes uses modes and matching dissimilarity.
- Hierarchical clustering is read by dendrogram height.
- GMM is soft clustering fitted by EM.
- Neural networks need scaling and learn by backpropagation.

End of Reference - Topics 1-27 covering Weeks 8-13